main.py

Visualize | Run ▶

```python

import numpy as np

# Marks of 4 students in Math, Science, English, History
student_scores = np.array([
    [85, 90, 78, 88],
    [92, 85, 80, 75],
    [76, 88, 92, 84],
    [89, 95, 85, 90]
])

# Calculate average marks for each subject
average_scores = np.mean(student_scores, axis=0)

subjects = ["Math", "Science", "English", "History"]

# Display average marks
for subject, avg in zip(subjects, average_scores):
    print(subject, "Average:", round(avg, 2))

# Find subject with highest average
highest_index = np.argmax(average_scores)
print("Highest Average Subject:", subjects[highest_index])
print("Highest Average:", round(average_scores[highest_index], 2))
```

Output

```
Math Average: 85.5
Science Average: 89.5
English Average: 83.75
History Average: 84.25
Highest Average Subject: Science
Highest Average: 89.5
```

main.py

Visualize | Run ▶

```
1
2  import numpy as np
3
4  # Sales data for 3 products over 3 sales
5  sales_data = np.array([
6      [100, 150, 200],
7      [250, 300, 350],
8      [120, 180, 240]
9  ])
10
11 # Calculate average price of all products sold
12 average_price = np.mean(sales_data)
13
14 print("Average Price:", average_price)
15
```

Output

```
Average Price: 210.0
```

main.py

Visualize | Run ▶

Output

```
Average Sale Price: 400000.0
```

```python

import numpy as np

# Columns: Bedrooms, Square Footage, Sale Price
house_data = np.array([
    [3, 1200, 200000],
    [5, 1800, 350000],
    [4, 1500, 280000],
    [6, 2200, 450000],
    [5, 2000, 400000]
])

# Filter houses with more than 4 bedrooms
filtered_houses = house_data[house_data[:, 0] > 4]

# Calculate average sale price
average_price = np.mean(filtered_houses[:, 2])

print("Average Sale Price:", average_price)

```

Programiz
Python Online Compiler

Visualize | Run ▶

main.py

```
import numpy as np

# Quarterly sales data
sales_data = np.array([10000, 12000, 15000, 18000])

# Calculate total annual sales
total_sales = np.sum(sales_data)

# Calculate percentage increase from Q1 to Q4
percentage_increase = (
    (sales_data[3] - sales_data[0]) / sales_data[0]
) * 100

print("Total Annual Sales:", total_sales)
print("Percentage Increase:", percentage_increase, "%")

```

Output

```
Total Annual Sales: 55000
Percentage Increase: 80.0 %
```

Programiz
Python Online Compiler

Visualize | Run ▶

main.py

```
import numpy as np

# Fuel efficiency of car models in MPG
fuel_efficiency = np.array([25, 30, 35, 40, 45])

# Calculate average fuel efficiency
average_efficiency = np.mean(fuel_efficiency)

# Compare two car models
car1 = fuel_efficiency[0]
car2 = fuel_efficiency[4]

# Calculate percentage improvement
improvement = ((car2 - car1) / car1) * 100

print("Average Fuel Efficiency:", average_efficiency, "MPG")
print("Percentage Improvement:", improvement, "%")
```

Output

```
Average Fuel Efficiency: 35.0 MPG
Percentage Improvement: 80.0 %
```

Programiz
Python Online Compiler

IC Trade on MT4 with Raw Pricing
Ultra-low latency execution
Start Trading

main.py

Visualize | Run ▶

```python

# Item prices and quantities
prices = [100, 200, 150]
quantities = [2, 1, 3]

# Discount and tax rates
discount_rate = 10
tax_rate = 5

# Calculate subtotal
subtotal = sum(p * q for p, q in zip(prices, quantities))

# Apply discount
discount = subtotal * discount_rate / 100
discounted_amount = subtotal - discount

# Apply tax after discount
tax = discounted_amount * tax_rate / 100
total_cost = discounted_amount + tax

print("Subtotal:", subtotal)
print("Discount:", discount)
print("Tax:", tax)
print("Total Cost:", total_cost)

```

Output

```
Subtotal: 850
Discount: 85.0
Tax: 38.25
Total Cost: 803.25
```